PRÁCTICA CALIFICADA 02

Avance del Proyecto: Sócrates Bot

Grupo 1	
Integrantes	Luis Alanya Campos Jharvy Cadillo Tarazona Pedro Vargas Alfaro

1. Fundamento y Cumplimiento de Requerimientos

Sócrates Bot es una Aplicación Web Progresiva (PWA) diseñada para reducir la tasa de abandono en estudiantes iniciales de programación en Python e incluir de forma plena a estudiantes con discapacidad motora severa o discapacidad visual. A diferencia de los asistentes generativos convencionales que entregan el código resuelto, Sócrates Bot aplica el método socrático guiado por voz: analiza el razonamiento del alumno, formula preguntas orientadoras y vincula cada logro algorítmico con el bienestar y evolución de una mascota virtual (pingüino).

Requerimiento	Implementación	Usabilidad
1. Interfases de Voz	Reconocimiento de voz (Whisper STT + Web Speech API) con gramática de dictado Python en español y síntesis TTS + earcons.	Control 100% libre de manos (Hands-Free) para discapacidad motora y operación auditiva para ceguera.
2. LLM con Capa de Abstracción	Patrón Strategy/Adapter (LLMRouter) que conmuta entre API remota (Gemini/OpenAI) y LLM local vía Ollama.	Failover automático en < 5 s ante caída de red sin interrumpir el diálogo socrático.
3. Plataforma PWA	Manifiesto instalable (manifest.json), Service Worker (sw.js) y persistencia en IndexedDB.	Ejecución en laptop y móvil con preservación del progreso ante desconexiones.
4. Personalización Continua	Perfil persistente que registra ritmo de respuesta, errores por tema, preferencias TTS y emociones inferidas.	Ajuste dinámico de dificultad, tiempo de escucha del micrófono y tono de las pistas.
5. Sistema Multiusuario	Salas colaborativas en tiempo real (WebSockets) para co-resolver código y visitar mascotas del grupo.	Aprendizaje social entre usuarios y reducción del aislamiento estudiantil.
6. Diseño Emocional	Niveles Visceral (estética y audio amigable), Conductual (feedback < 300 ms) y Reflexivo (evolución de mascota).	Transforma el error de sintaxis en un acompañamiento empático libre de ansiedad.

2. Análisis de Stakeholders

Se tomó en cuenta cinco perfiles de usuarios e interesados que intervienen en el sistema, evaluando sus diferencias físicas, sensoriales y cognitivas y cómo estas determinan el diseño de la interfaz:

Stakeholder	Características y Necesidades Centrales	Diferencias Clave	Cómo Influye en el Diseño de Sócrates Bot
SH-01: Estudiantes Novatos	Alumnos en etapa inicial de lógica y sintaxis Python, sufren bloqueo ante errores crípticos.	Requieren motivación constante y guía paso a paso.	Mascota evolutiva gamificada y pistas socráticas graduales que no regalan el código.
SH-02: Estudiantes con Discapacidad Motora	Limitaciones severas en miembros superiores, no pueden usar teclado físico ni ratón.	Entrada exclusivamente vocal con posibles pausas respiratorias.	Comandos de voz equivalentes para el 100% de la GUI y dictado estructurado de sintaxis e indentación Python.
SH-03: Estudiantes con Discapacidad Visual	Usuarios con ceguera total o baja visión severa que necesitan lectores de pantalla.	Procesamiento auditivo secuencial.	Cumplimiento WCAG 2.1 AA, atributos WAI-ARIA (aria-live) y audiodescripción.
SH-04: Profesores / Tutores	Docentes encargados de supervisar el progreso y detectar alumnos en riesgo.	Enfoque analítico grupal y detección de cuellos de botella.	Dashboard Docente con métricas de avance, errores frecuentes y alertas de bloqueo.
SH-05: Desarrolladores	Ingenieros responsables del backend, seguridad del sandbox y modelos LLM.	Enfoque en estabilidad, concurrencia y desacoplamiento.	Mejoramiento del sistema en base a feedbacks

3. Análisis PACT y Requerimientos

El marco PACT (Personas, Actividades, Contextos y Tecnologías) permite vincular las tareas reales de los usuarios con las restricciones tecnológicas del sistema:

Actividad PACT	Análisis	Requerimientos del Diseño
Navegación del Sistema	Entornos domésticos o móviles; usuarios sin ratón o sin visión espacial de la lista de ejercicios.	Comandos de voz directos ('Sócrates, seleccionar ejercicio tres') y lectura automática WAI-ARIA del estado de cada nodo.

Actividad PACT	Análisis	Requerimientos del Diseño
Dictado Estructurado y Edición de Código Python	Python exige indentación exacta de 4 espacios y dos puntos (:), el micrófono puede captar ruido de fondo.	Parseo semántico de estructuras Python habladas, confirmación TTS y rechazo seguro de audio ambiguo con earcons.
Diálogo Socrático y Depuración de Errores	Alto riesgo de frustración tras errores consecutivos, necesidad de latencia conversacional fluida.	Intervención empática automática al 3.er fallo consecutivo con pregunta orientadora en <= 5.0 segundos.
Vínculo Emocional con la Mascota Evolutiva	Conservación de la motivación a lo largo del curso para todos los perfiles.	Tres niveles de Norman (visceral, conductual, reflexivo) con evolución de Bebé a Aprendiz/Sabio narrada por TTS.
Colaboración en Salas y Continuidad en Red Inestable	Trabajo grupal remoto y posibles caídas de internet o de la API externa del LLM.	Salas multiusuario WebSockets, persistencia offline en PWA y conmutación a LLM local en < 5 s.

4. Requerimientos Funcionales y No Funcionales

4.1. Requerimientos Funcionales (RF-01 a RF-08)

ID	Nombre	Requerimiento Funcional	Prioridad
RF-01.1	Orientación Socrática sin Solución Directa	El sistema debe guiar al estudiante mediante preguntas y pistas generadas por el LLM sin entregar nunca el código resuelto.	Alta
RF-01.2	Evaluación de Razonamiento por Voz	El LLM debe evaluar las explicaciones verbales y conceptuales del alumno para adaptar el nivel de ayuda de forma personalizada.	Alta
RF-01.3	Mascota Evolutiva según Progreso	La GUI debe actualizar progresivamente la fase de la mascota (Huevo -> Bebé -> Aprendiz -> Sabio) según los logros en Python.	Alta

ID	Nombre	Requerimiento Funcional	Prioridad
RF-02.1	Comandos de Voz para el 100% de la GUI	Todas las acciones de navegación, selección, ejecución y salas deben poseer un comando de voz equivalente sin teclado ni ratón.	Alta
RF-02.2	Dictado y Control de Sintaxis Python	El módulo STT debe interpretar instrucciones verbales para escribir estructuras Python, símbolos e indentación automática.	Alta
RF-03.1	Compatibilidad con Lectores de Pantalla	La interfaz debe incluir roles semánticos, etiquetas accesibles y regiones aria-live compatibles con NVDA, JAWS y VoiceOver.	Alta
RF-03.2	Voz Bidireccional Integrada (TTS)	El sistema debe narrar por voz el feedback del LLM, el código con su nivel de indentación y las expresiones de la mascota.	Alta
RF-04.1	Dashboard Docente de Progreso	El sistema debe ofrecer a los profesores un módulo analítico con ejercicios completados, logros, errores frecuentes y bloqueos.	Media
RF-05.1	Orquestación Backend y Sandbox Python	El backend debe orquestar STT/TTS, LLM y la ejecución aislada de código Python en contenedores seguros sin cruce de datos.	Alta
RF-06.1	Personalización Continua Persistente	El sistema debe actualizar continuamente el perfil del usuario con su ritmo, tasa de errores, preferencias y emociones inferidas.	Alta

ID	Nombre	Requerimiento Funcional	Prioridad
RF-07.1	Salas Multiusuario Colaborativas	La aplicación debe permitir crear y unirse a salas en tiempo real para co-resolver retos y visitar las mascotas de compañeros.	Media
RF-08.1	Diseño Emocional de Norman	El sistema debe implementar respuestas viscerales, conductuales y reflexivas con micro-interacciones y tono afectivo de apoyo.	Alta

4.2. Requerimientos No Funcionales (RNF-01 a RNF-05)

ID Req	Categoría	Descripción Formal y Métrica de Verificación	Prioridad
RNF-01.1	Estándar WCAG 2.1 Nivel AA	Cumplimiento de pautas WCAG 2.1 AA y WAI-ARIA con contraste >= 4.5:1 y cero violaciones críticas en auditoría Axe/Lighthouse.	Alta
RNF-01.2	Tolerancia en Reconocimiento STT	Alta tolerancia a pausas de hasta 3.5 s y variaciones en el ritmo de habla, previniendo ejecuciones accidentales ante ruido.	Alta
RNF-02.1	Latencia Conversacional <= 5 s	El flujo completo STT -> LLM -> TTS no debe superar los 5.0 segundos con hasta 50 estudiantes conectados simultáneamente.	Alta
RNF-03.1	Capa de Abstracción LLM (< 1.8 s)	Conmutación desacoplada automática entre API remota y modelo local de respaldo (Ollama) en menos de 5 segundos.	Alta

ID Req	Categoría	Descripción Formal y Métrica de Verificación	Prioridad
RNF-04.1	Plataforma PWA Offline-First	Ejecución instalable en laptops y móviles con almacenamiento local (IndexedDB) ante caídas de red.	Media
RNF-05.1	Sandbox	Ejecución aislada con timeout de 2.0 s por script.	Media

5. Historias de Usuario y BDD

HU-01: Navegación y Resolución de Ejercicios por Voz (Accesibilidad Total)

Como estudiante con discapacidad motora o visual, Quiero navegar por el sistema y dictar código Python mediante comandos de voz (STT), Para resolver los ejercicios de forma autónoma sin depender del teclado ni del ratón.

- **Escenario 1 (Éxito BDD):** Given que el usuario esté navegando por la aplicación con STT activo; When dice 'Sócrates, seleccionar ejercicio tres' y dicta 'definir función suma con parámetros a y b dos puntos retornar a más b'; Then el sistema escribe 'def suma(a, b):\n   return a + b' con indentación correcta y confirma por TTS.
- **Escenario 2 (Alternativo / Error BDD):** Given un entorno con ruido o dicción ambigua; When el STT obtiene baja confianza (<0.65); Then no altera el editor, emite un earcon de advertencia y pide por TTS repetir o decir 'Ayuda de comandos'.

HU-02: Retroalimentación Emocional y Accesible de la Mascota Evolutiva

Como estudiante novato de programación, Quiero visualizar y escuchar los cambios emocionales y de evolución de la mascota virtual según mi desempeño, Para mantenerme motivado y comprender intuitivamente mi progreso.

- **Escenario 1 (Éxito BDD):** Given que la mascota está en estado 'Bebé' y el alumno supera el ejercicio 5; When el motor registra el hito; Then la GUI anima la evolución a 'Aprendiz', actualiza aria-live y el TTS narra: '¡Felicidades! Tu mascota ha evolucionado a Aprendiz porque dominaste los bucles'.
- **Escenario 2 (Alternativo / Error BDD):** Given que el usuario falla 3 veces consecutivas un ejercicio; When el intérprete devuelve el 3.er error; Then la mascota pasa a estado reflexivo/apoyo, emite alternativa textual accesible y el LLM genera una pista guiada sin dar la solución.

HU-03: Gestión de Sesiones Concurrentes y Conmutación en Capa de Abstracción LLM

Como desarrollador/administrador, Quiero que el backend gestione peticiones concurrentes en contenedores aislados y conmute entre LLM remoto y local, Para garantizar estabilidad, aislamiento y respuestas en tiempo real.

- **Escenario 1 (Éxito BDD):** Given 50 estudiantes conectados simultáneamente; When envían código o voz; Then cada solicitud corre en un sandbox aislado (< 2.0 s) y responde en menos de 5.0 segundos sin cruce de sesiones.
- **Escenario 2 (Alternativo / Error BDD):** Given una caída o saturación de la API remota del LLM; When un alumno solicita tutoría; Then la capa de abstracción conmuta al LLM local (Ollama) en < 1.8 s o guarda el progreso localmente en la PWA.

HU-04: Salas Multiusuario Colaborativas

Como estudiante de programación, Quiero unirme por voz o clic a salas colaborativas para resolver retos en pareja y visitar mascotas de compañeros, Para aprender y compartir conocimientos en grupo.

- **Escenario 1 (Éxito BDD):** Given dos alumnos en la Sala #104; When el Alumno A dicta código y cede el turno por voz; Then el WebSocket sincroniza la pizarra en < 300 ms, narra por TTS el cambio al Alumno B y el tutor modera la co-resolución.
- **Escenario 2 (Alternativo / Error BDD):** Given una desconexión de un compañero en sala; When supera 10 s sin respuesta; Then el sistema preserva el código compartido y permite continuar asistido por la mascota virtual.

HU-05: Personalización Continua del Perfil Cognitivo y Afectivo

Como estudiante recurrente, Quiero que el sistema actualice continuamente mi perfil con mi ritmo, errores frecuentes y estado emocional, Para recibir una experiencia adaptada en cada sesión.

- **Escenario 1 (Éxito BDD):** Given un perfil con 45% de errores previos en listas y preferencia TTS 1.25x; When inicia sesión; Then el sistema aplica automáticamente sus ajustes de accesibilidad y propone un reto de refuerzo personalizado.
- **Escenario 2 (Alternativo / Error BDD):** Given que el sistema detecta frustración declarada o aumento brusco de errores; When actualiza el estado afectivo; Then simplifica las pistas socráticas y la mascota sugiere una pausa breve o sub-paso guiado.

HU-06: Supervisión Docente y Analíticas de Aprendizaje

Como profesor/tutor, Quiero consultar en un dashboard el progreso, errores por tema y alertas de bloqueo de cada estudiante, Para detectar dificultades a tiempo y evitar retiros del curso.

- **Escenario 1 (Éxito BDD):** Given que el docente ingresa al Dashboard Docente (Pantalla P6); When filtra su grupo; Then visualiza ejercicios completados, mapa de calor de errores y alumnos con 3+ bloqueos, pudiendo exportar el reporte.
- **Escenario 2 (Alternativo / Error BDD):** Given un grupo nuevo sin sesiones; When consulta analíticas; Then permite cargar datos simulados de validación sin producir fallos en la vista.

6. Matriz de Trazabilidad Completa

ID	Descripción	Tipo	Prioridad	Historia(s) de Usuario que toma	ID HU	Estado	Evidencia / Criterio de Aceptación
RF-01.1	Pistas socráticas del LLM sin dar la solución directa.	FR	Alta	Retroalimentación Emocional y Accesible de Mascota	HU-02	---	Escenario 2: Ante 3 fallos consecutivos, la mascota pasa a modo apoyo y el LLM da una pista sin código explícito.
RF-01.2	Evaluación del razonamiento dictado por el alumno.	FR	Alta	Navegación por Voz / Personalización Continua	HU-01, HU-05	----	Escenario 1: El sistema procesa la explicación por voz y adapta la dificultad de la ayuda según la lógica demostrada.
RF-01.3	Evolución de la mascota según logros pedagógicos.	FR	Alta	Retroalimentación Emocional y Accesible de Mascota	HU-02	----	Escenario 1: Completar el ejercicio 5 activa la animación y narración de cambio de fase 'Bebé' a 'Aprendiz'.
RF-02.1	Comandos de voz equivalentes para el 100% de la GUI.	FR	Alta	Navegación y Resolución de Ejercicios por Voz	HU-01	----	Escenario 1: Comando 'Sócrates, seleccionar ejercicio tres' navega al nodo sin requerir ratón ni teclado.
RF-02.2	Dictado y parseo estructurado de sintaxis Python.	FR	Alta	Navegación y Resolución de Ejercicios por Voz	HU-01	----	Escenario 1: Dictar 'definir función suma...' escribe def suma(a, b): con la indentación correcta en el editor.
RF-03.1	Compatibilidad con lectores de pantalla y WAI-ARIA.	FR	Alta	Retroalimentación Emocional y Accesible de Mascota	HU-02	----	Escenario 1: Cambios de estado actualizan atributos aria-live para que NVDA/JAWS/VoiceOver anuncien el evento.
RF-03.2	Narración por voz (TTS) del feedback, código y mascota.	FR	Alta	Navegación por Voz / Mascota Accesible	HU-01, HU-02	----	Escenario 1: La síntesis TTS narra la evolución de la mascota y lee el nivel de indentación y feedback socrático.

ID	Descripción	Tipo	Prioridad	Historia(s) de Usuario que toma	ID HU	Estado	Evidencia / Criterio de Aceptación
RF-04.1	Dashboard docente de avance, logros y errores.	FR	Media	Supervisión Docente y Analíticas de Aprendizaje	HU-06	----	Escenario 1: Consulta de ejercicios completados, mapa de calor de errores y detección de alumnos en bloqueo.
RF-05.1	Backend multiusuario, STT/TTS y sandbox de Python.	FR	Alta	Gestión de Sesiones Concurrentes y Sandbox	HU-03	----	Escenario 1: Aislamiento de código en contenedor seguro y procesamiento de peticiones concurrentes en < 2.0 s.
RF-06.1	Personalización continua del perfil persistente.	FR	Alta	Personalización Continua del Perfil Cognitivo/Afectivo	HU-05	----	Escenario 1 y 2: Actualización automática de ritmo, errores frecuentes y estado emocional inferido.
RF-07.1	Salas multiusuario colaborativas y visitas sociales.	FR	Media	Salas Multiusuario Colaborativas y Visitas	HU-04	----	Escenario 1: Co-resolución sincrónica en pizarra compartida y visualización/narración de mascotas del grupo.
RF-08.1	Diseño emocional de Norman (visceral, conductual, reflexivo).	FR	Alta	Mascota Evolutiva / Personalización Afectiva	HU-02, HU-05	----	Escenario 1 y 2: Micro-interacciones afectivas, tono empático del asistente y evolución reflexiva del Búho.
RNF-01.1	Cumplimiento del estándar WCAG 2.1 Nivel AA.	NFR	Alta	Retroalimentación Emocional y Accesible de Mascota	HU-02	----	Cero violaciones críticas en auditoría (Axe/Lighthouse), contraste >= 4.5:1 y lectura auditiva completa.

ID	Descripción	Tipo	Prioridad	Historia(s) de Usuario que toma	ID HU	Estado	Evidencia / Criterio de Aceptación
RNF-01.2	Tolerancia del STT a pausas y variaciones de habla.	NFR	Alta	Navegación y Resolución de Ejercicios por Voz	HU-01	----	Escenario 2: Si el audio es ambiguo, el sistema emite un earcon sonoro y pide reintentar por TTS sin dañar el código.
RNF-02.1	Latencia conversacional STT-LLM-TTS <= 5 segundos.	NFR	Alta	Gestión de Sesiones Concurrentes y Sandbox	HU-03	----	Escenario 1: Respuestas de voz fluidas en <= 5.0 s con 50 usuarios simultáneos conectados.
RNF-03.1	Capa desacoplada para conmutación de modelos LLM.	NFR	Alta	Gestión de Sesiones Concurrentes y Sandbox	HU-03	----	Escenario 2: Ante caída de la API remota, el sistema conmuta al LLM local (Ollama) en menos de 5 s.
RNF-04.1	Arquitectura PWA multiplataforma con modo offline.	NFR	Media	Gestión de Sesiones / Personalización Continua	HU-03, HU-05	----	Escenario 2: Guardado local en IndexedDB y ejecución instalable en laptops y dispositivos móviles.
RNF-05.1	SandBox.	NFR	Media	Gestión de Sesiones Concurrentes en Backend	HU-03	----	Timeout de ejecución de 2.0 s.

7. Sistemas y Aplicaciones Similares

Se evaluaron cuatro plataformas similares (Khanmigo, Duolingo, Mimo y Talon Voice) para identificar brechas de accesibilidad e interacción:

Sistema Analizado	Fortalezas Principales	Limitaciones frente a Nuestro Contexto	Ventaja Superadora de Sócrates Bot
Khanmigo (Khan Academy)	Excelente guía socrática basada en LLM sin dar respuestas directas.	Interfaz centrada en texto/teclado; sin mascota evolutiva ni programación hands-free.	Extiende el método socrático al canal de voz bidireccional con mascota afectiva.

Sistema Analizado	Fortalezas Principales	Limitaciones frente a Nuestro Contexto	Ventaja Superadora de Sócrates Bot
Duolingo	Gamificación altamente efectiva, rachas y mascota expresiva (Duo).	Ejercicios cerrados de idiomas, no permite dictar ni depurar código Python libre.	Vincula la evolución de la mascota al razonamiento algorítmico real en Python.
Mimo (Learn to Code)	Ruta progresiva de micro-lecciones de Python en móvil y web.	Depende de interacción visual y táctil, inaccesible para discapacidad motora severa o ceguera.	PWA Open Source (MIT) con dictado estructurado por voz y lectura WAI-ARIA.
Talon Voice	Control total por voz hands-free para programadores con discapacidad motora.	Curva de aprendizaje técnica extrema (comandos fonéticos estrictos), sin tutoría ni gamificación.	Dictado en español natural pensado para novatos con acompañamiento socrático.

8. Criterios de Usabilidad

ID	Criterio de Usabilidad	Métrica de Aceptación
CU-01	Hands-Free (Accesibilidad Motora)	100% de las pantallas y ejercicios completables usando exclusivamente comandos de voz.
CU-02	Visibilidad del Estado del Sistema Multimodal	Feedback visual, sonoro y WAI-ARIA en <= 300 ms tras cada comando.
CU-03	Eficiencia en Dictado de Sintaxis Python	Reducción >= 60% en sílabas pronunciadas en comparación a deletreo manual mediante comandos de voz predefinidos.
CU-04	Prevención y Recuperación de Errores	0% de corrupción en el editor ante ruido (confianza STT < 0.65) y comando 'deshacer' en un paso.
CU-05	Comprensión Auditiva No Visual (WCAG 2.1 Nivel AA)	Puntaje >= 95/100 en Lighthouse Accessibility con lectura verbal del nivel de indentación.

9. Tres Alternativas de Diseño, Prototipo Óptimo y Diagrama de Interacciones

Se diseñaron y evaluaron tres alternativas arquitectónicas de interfaz antes de construir la simulación de fidelidad media:

Alternativa de Diseño Propuesta	Descripción de la Estructura de Interacción	Puntaje Multicriterio y Decisión
Alternativa 1: IDE Clásico Adaptado	Árbol de archivos lateral, menús desplegables, editor central denso y botón de micrófono flotante.	5 / 10 — Descartada (Alta carga cognitiva en novatos y difícil navegación por voz).
Alternativa 2: Interfaz Conversacional Pura	Flujo único de burbujas de chat donde se mezclan los mensajes del tutor y los fragmentos de código.	7 / 10 — Descartada (Pérdida de contexto espacial e indentación en scripts de más de 6 líneas).
Alternativa 3: Workspace Dual-Modal Socrático (ÓPTIMA)	Panel izquierdo afectivo (Mascota + Tutor Socrático), panel derecho de código estructurado por voz y barra inferior permanente de comandos de voz visibles.	9 / 10 — Seleccionada (Equilibrio entre guía emocional de Norman, claridad espacial Python y accesibilidad WAI-ARIA).

10. Diagrama de Gantt del Proyecto

